

Table of Contents

1. Task 01: Introduction.....	1
1.1. Introduction to security	1
1.2. Introduction to CIA triad	1
1.3. Introduction to Cryptography	2
1.4. Key terminologies.....	3
2. Task 02: Background	5
2.1. Brief history of cryptography.....	5
2.2. Symmetric (secret-key) encryption.....	6
2.3. Asymmetric (public-key) encryption	7
2.4. Caesar cipher and known weakness.....	7
2.4.1. Encrypting and decrypting the word “Apple” using Caesar cipher	8
2.5. Rationale for modification.....	9
3. Task 03: Development of a new Cryptographic Algorithm	10
3.1. Algorithm Name	10
3.2. Design choices.....	10
3.3. Text encoding (ASCII/UTF-8) and hexadecimal representation	11
3.4. XOR Logic Gate	14
3.5. AND Logic Gate	15
3.6. Encryption steps (per byte)	15
3.7. Decryption steps (reverse process).....	17
3.8. Algorithm explanation.....	19
3.9. Worked example for encryption.....	20
4. Task 04: Example test cases	22
4.1. Encryption Process	22
4.1.1. Test Case 1: “Samriddha”.....	22
4.1.2. Test Case 02: “Security”	23
4.1.3. Test Case 03: “CIA-TRIAD”	23
4.1.4. Test Case 04: “Attack at dawn!”	24
4.1.5. Test Case 05: “12345”	25
4.2. Decryption Process	26

4.2.1.	Test Case 1: “Samriddha”	26
4.2.2.	Test Case 02: “Security”	26
4.2.3.	Test Case 03: “CIA-TRIAD”	27
4.2.4.	Test Case 04: “Attack at dawn!”	28
4.2.5.	Test Case 05: “12345”	29
5.	Task 05: Critical Evaluation of the LGCC Algorithm	30
5.1.	Strengths of LGCC	30
5.2.	Weaknesses of LGCC	31
5.3.	Application Area of LGCC	33
6.	Conclusion	34
7.	References	35
8.	Appendices	36
8.1.	Appendix A: LGCC Supporting Artefacts	36
8.1.1.	Variable definition	36
8.1.2.	Bit rotation functions	36
8.2.	Appendix B: LGCC Pseudocode	36
8.2.1.	Encryption and Decryption Pseudocode	36

List of Figures

Figure 1: Flowchart of Encryption process.	16
Figure 2: Flowchart of the Decryption process.	18
Figure 3: Per-byte processing pipeline used by LGCC, including the key-derived values that are recomputed for every index i.	19
Figure 4: XOR mixing.	19

List of Tables

Table 1: ASCII/UTF 8 decimal and hexadecimal values for characters in the word "HELLO" (Rapidtables, n.d.).....	12
Table 2: Table of hexadecimal digits and their decimal values (Byjus, n.d.).	13
Table 3: XOR Gate truth table (Geeksforgeeks, 2025).....	14
Table 4: AND gate truth table (Geeksforgeeks, 2025).....	15
Table 5: Full encryption trace for HELLO using KEY.....	20
Table 6: Encryption table of "Samriddha".....	22
Table 7: Encryption table of "Security".	23
Table 8: Encryption table of "CIA-TRIAD".	24
Table 9: Encryption table of "Attack at dawn!".....	25
Table 10: Encryption table of "12345".	25
Table 11: Decryption Table of "Samriddha".	26
Table 12: Decryption Table of "Security".....	27
Table 13: Decryption Table of "CIA-TRIAD".....	28
Table 14: Decryption Table of "Attack at dawn!".	28
Table 15: Decryption Table of "12345".....	29

Abstract

This coursework was completed to strengthen understanding of core information security concepts and how cryptography supports the Confidentiality, Integrity, and Availability (CIA) triad in real computing environments. It is relevant to the reader because it connects foundational theory (security goals and cryptographic principles) to a practical design exercise showing how an insecure classical cipher can be systematically improved using modern byte-level operations.

The issue is that the basic Caesar cipher still has weak security because of the limited patterns it presents. This means that it is still extremely vulnerable to brute force and frequency analysis attacks. Therefore, the central focus of the project has been on how to improve the algebraic methodology approach based on Caesar ciphers while still having multiple layers of security, and still keeping the encryption and decryption reversible and straightforward.

In the case of the coursework, the first step was to outline some of the basics of security and basic cryptography, followed by the analysis of the Caesar cipher and its shortcomings. The next step was to design and build a new type of symmetry encryption, and for the separation of the components of the encryption device, the device was called the Logic Gate Caesar Cipher (LGCC). This device works on the division of data into bytes and combines modular shifting with a selective level of mixing based on XOR, some level of feedback with the ciphertext (or a feedback mechanism that is based on chaining), and 8-bit rotation, all of which improves diffusion and decreases repetition of ciphered text. The algorithm was built for a complete example, and a series of test cases to demonstrate that the encryption scheme is not a one-to-one mapping (a property that is highly desirable for reversible schemes), and furthermore the algorithm was constructed to ensure that the ciphertext was always capable of being reproduced to exactly the original text.

The outcomes show that LGCC is functionally correct, reversible, and produces ciphertext that is less pattern-revealing than a basic Caesar cipher. In terms of implications for information security, this project highlights why classical substitution ciphers are unsuitable for real protection and demonstrates how layering diffusion, key-based mixing, and feedback can significantly strengthen confidentiality. At the same time, it reinforces the broader security lesson that custom ciphers require careful evaluation and should be used mainly for educational and learning-oriented purposes rather than high-assurance deployments.

1. Task 01: Introduction

1.1. Introduction to security

The term security in the context of information systems refers to the protection of information and systems against access, use, disclosure, disruption, modification, or destruction without permission. The application of security controls is to protect the reliability and trustworthiness of information and services for legitimate users. NIST defines this objective using the protection of confidentiality, integrity, and availability (National Institute of Standards and Technology (NIST), 2017), (National Institute of Standards and Technology (NIST), 2004).

1.2. Introduction to CIA triad

The CIA triad is a foundational model used to describe what “quality” information security must achieve: Confidentiality, Integrity, and Availability. NIST defines computer security as protection that preserves these objectives for information system resources (National Institute of Standards and Technology (NIST), 2004).

Confidentiality protects data so that only those who have rights to access it and the processes that work on the data are able to view and use it. When and how data are kept and transferred are often the weak links. Exposed data in any of those avenues cause data breaches and insider threats. Confidentiality controls are typically accomplished by employing some form of identification, authentication processes, and encryption protocols (National Institute of Standards and Technology (NIST), 2017).

Integrity protects data so that only those who have the authority to make changes to the data can alter it. Integrity breaches may occur because records are modified and access to some of the files is tampered. Also, the software or the settings may have some malicious changes. Integrity controls may include use of secure hashes, logging user activities, controlling access to files, and utilizing some cryptographic method like digital signatures (National Institute of Standards and Technology (NIST), 2004).

Availability allows access to data, applications or any system to the user or user group that has been authorized to do so. Availability of data or applications may be disrupted on purpose using denial of service attacks or may occur inadvertently via hardware failure, misconfiguration, or use of malware. Availability controls may include use of redundancy, data backups, system and network monitoring, and disaster recovery (National Institute of Standards and Technology (NIST), 2017).

Understanding the CIA triad is of critical importance because real world security incidents will target, or in some way damage, all or some of the triad. A system that is designed to protect all the components of the CIA triad in a contextual and risk aware manner is the most secure.

1.3. Introduction to Cryptography

Cryptography is used to protect information by transforming readable data (plaintext) into an unreadable form (ciphertext) using an encryption algorithm and a key. Information becomes secure through cryptography, which changes plain text into scrambled code via algorithms together with keys. When received correctly, only authorized users unlock such encoded messages. Decryption returns these protected contents back to their initial state. This process ensures confidentiality during transmission across systems. A key cryptographic service is encryption, where readable data (plaintext) is transformed into an unreadable form (ciphertext) to conceal meaning and reduce unauthorised use. If the transformation is reversible,

decryption restores the ciphertext back to its original plaintext (National Institute of Standards and Technology (NIST), n.d.).

This milestone documents background research on cryptography and introduces the design of a new cipher that builds on the Caesar cipher. The new design adds additional mathematical operations and logic-gate operations to reduce the patterns and weaknesses that occur in a basic substitution cipher.

1.4. Key terminologies

Some of the key terminologies used in cryptography are:

- Plain text: The plaintext includes the original readable message/data before being encrypted.
- Cipher text: The ciphertext is the unreadable form of the data after it is encrypted. Hence, it cannot be readable without the right key.
- Key: The key is a value or a set of values and is the reason for which transformation of plaintext to ciphertext is possible. Also, the key is the reason why ciphertext can be made readable.
- Encryption: When data is transformed into ciphertext for the purpose of hiding the original data, it is called encryption (National Institute of Technology, n.d.).
- Decryption: When the encryption is a reversible process and the original data is made readable, the process of transformation is called decryption.

- Symmetric cryptography: When a cryptographic algorithm uses a same key for encryption and decryption, it is called symmetric cryptography (National Institute of Technology , n.d.).
- Asymmetric cryptography: It uses a pair of related keys, a public and a private. The public key is used for encrypting or signing. The private key is used for decryption or verification (National Institute of Technology , n.d.).
- Brute-force attack: When many possible keys are used in an attempt to find the right one, it is called a brute-force attack. Thus, small key spaces are weak.
- Frequency analysis: When language patterns remain visible in weak substitution ciphers, it is referred to as frequency analysis.

2. Task 02: Background

2.1. Brief history of cryptography

Cryptography developed through multiple historical phases. Encyclopaedia Britannica identifies three well-defined phases in the history of cryptology: early manual cryptography (origin through World War I), followed by later mechanised approaches, and then modern information-age cryptology (Britannica, 2025).

In both Ancient and classical history, we can find examples of the early attempts at defending the communication of military and politicians. In Ancient Greece, the Spartan scytale was a device consisting of a cylinder and a strip of parchment wrapped around it in which a message was inscribed. When the parchment was unwound, the letters became unscrambled and could be read only by someone possessing a scytale of the same diameter (Britannica, 2025). The Caesar Cipher is another classical cipher of great historical significance. It applies a fixed number of shifts of the letters in the text of the message, which makes it easy and useful, and is one of the first ciphers someone learns, but it is also a very weak cipher since it can easily be broken through brute force and frequency analysis (Britannica , 2025).

Modern cryptography has moved beyond manual and mechanical methods into mathematically grounded schemes supporting secure communication at scale, including encryption, authentication, and digital signatures across internet systems.

2.2. Symmetric (secret-key) encryption

Symmetric cryptography, which is also known as key cryptography uses the same key for encryption and for decryption. This way of doing things is usually computationally efficient. Symmetric cryptography is commonly used to encrypt amounts of data like files or large volumes of data. Symmetric cryptography is a choice, for this because it works well with large amounts of data (National Institute of Technology , n.d.), (National Institute of Standards and Technology (NIST), 2017).

Key challenge (key distribution):

The secret key has to be the same, for both parties. So, the key needs to be shared and kept in a safe place. This is a problem because the key has to be given to both parties without anyone else getting it. If the secret key is intercepted or leaked or used again in a way or not handled properly the whole system is not secure. It has to be protected (National Institute of Technology , n.d.).

Appropriate use cases

When talking about encryption it is appropriate when two parties already have a safe way to share a secret key. The cause of this is they need to have this key to make it work. They can use another method to set up the key first and then use symmetric encryption. Symmetric encryption is very useful, in these situations because symmetric encryption relies on the key that the two people share.

2.3. Asymmetric (public-key) encryption

There are two keys in asymmetric or public key cryptography: a public key and a private key. Each key is used for a different role, meaning, for example, one key is used for signing, and the other for verifying (National Institute of Technology , n.d.).

Why it matters (less shared-secret key exchange):

An important benefit is the lesser exchange of a secret key. Meaning, the trust and the secure communication can be done without the need of a secret being shared between the two parties. That is why the public key is used for cryptographic systems.

Combining asymmetric with symmetric cryptography:

In most real-world applications, asymmetric systems are used with symmetric systems, where session keys are used, and then symmetric cryptography is used to secure the bulk of the information.

2.4. Caesar cipher and known weakness

Each letter in the plaintext moves forward a set number of places through the alphabet under the Caesar cipher, making it one type of substitution method. Its straightforward design allows quick understanding and setup, so educators often use it when teaching basic encryption ideas. Though limited in real security, it remains common in early lessons on code systems (Britannica , 2025).

2.4.1. Encrypting and decrypting the word “Apple” using Caesar cipher

We know,

$$C = (P + K) \bmod 26$$

Where, C is the cipher text, P is plain text and K is the key value.

So, by applying this formula, we get:

- $A (0) = 0 + 3 = 3 \bmod 26 = D$
- $P (15) = 15 + 3 = 18 \bmod 26 = S$
- $P (15) = 15 + 3 = 18 \bmod 26 = S$
- $L (11) = 11 + 3 = 14 \bmod 26 = O$
- $E (4) = 4 + 3 = 7 \bmod 26 = H$

Encrypted text = DSSOH

Now, to decrypt, we know that

$$P = (C - K) \bmod 26$$

So, we get,

- $D (3) = 3 - 3 = 0 \bmod 26 = A$
- $S (18) = 18 - 3 = 15 \bmod 26 = P$
- $S (18) = 18 - 3 = 15 \bmod 26 = P$
- $O (14) = 14 - 3 = 11 \bmod 26 = L$
- $H (7) = 7 - 3 = 4 \bmod 26 = E$

Despite its simplicity, the Caesar cipher fails to provide real security due to two flaws. With just 25 shifts available for standard letters, guessing the right one becomes practical through exhaustive testing. Because each letter always maps to the same substitute, frequent symbols stay frequent in the output. When certain characters appear more often, their encrypted versions reveal similar trends. An observer noticing

these repetitions may deduce the offset without needing any special tools. Patterns like these expose the hidden structure behind the scrambled text.

Because of its flaws, the Caesar method fits best in teaching settings. To make it more secure, extra steps are needed - steps that rely on keys to add randomness and hide repeating structures.

2.5. Rationale for modification

A Caesar-based design remains useful for educational purposes because it is simple. Nevertheless, its weaknesses require additional operations that introduce key-based variation, reduce visible patterns, and increase diffusion. The new design therefore introduces logic-gate mixing, bit rotation, and ciphertext feedback to strengthen the output while remaining reversible.

3. Task 03: Development of a new Cryptographic Algorithm

3.1. Algorithm Name

The new cipher is named LGCC (Logic Gate Caesar Cipher). The design retains a Caesar-style modular shift but adds XOR mixing, AND masking, bit rotation for diffusion, and ciphertext feedback (chaining). The selected algorithm is a symmetric encryption system as it uses the same key for both the encryption and decryption.

3.2. Design choices

- Data unit: bytes (0 to 255) rather than letters (A to Z), so any text can be processed as UTF-8 bytes.
- Key: a passphrase converted into key bytes.
- Caesar base step: modular addition (a shift) on each plaintext byte.
- Logic-gate step: XOR mixes the shifted value with key material and feedback; XOR is reversible.
- Diffusion step: an 8-bit rotation spreads small changes across different bit positions.
- Chaining: each ciphertext byte influences the next encryption step to reduce repeating patterns.

3.3. Text encoding (ASCII/UTF-8) and hexadecimal representation

LGCC operates on bytes (numerical values) rather than alphabetic characters. For this reason, before any encryption takes place, raw text needs turning into numbers between zero and two hundred fifty-five. UTF-8 handles this transformation, mapping each character to at least one byte. Most familiar symbols - like uppercase and lowercase English letters or punctuation marks - map directly because they sit within the classic ASCII set. Even though UTF-8 supports far more characters than ASCII ever did, basic texts stay unchanged in numeric value after encoding. So, nothing gets altered beyond representation; the original meaning stays intact throughout conversion. What emerges is simply a format compatible with how the cipher manipulates data internally.

Though less spacious than decimals, hex offers a tighter way to show byte data. Representing values from zero to twenty-five-five, one byte fits neatly into just two hex symbols. Take the letter "A" - it holds value sixty-five in base ten, appearing as forty-one when converted to hex. Similarly, "H" stands at seventy-two normally, turning into forty-eight within hex format. This form keeps information intact without boosting protection or changing content. Instead, it simply reshapes how numbers appear. To follow every move in the process clearly, these references help match letters, bytes, and their hex pairs correctly.

Table 1 provides the ASCII/UTF-8 decimal and hexadecimal values for the specific characters used in the worked example (Rapidtables, n.d.). These fixed numeric values explain why, for example, the letter 'H' can be treated as the byte value 72 (decimal) or 0x48 (hex) during encryption.

Character	Decimal	Hex	Binary (8-bit)
'H'	72	0x48	01001000
'E'	69	0x45	01000101
'L'	76	0x4C	01001100
'O'	79	0x4F	01001111
'K'	75	0x4B	01001011
'Y'	89	0x59	01011001

Table 1: ASCII/UTF 8 decimal and hexadecimal values for characters in the word "HELLO" (Rapidtables, n.d.).

Decimal	Hex digit	Binary (4-bit)
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Table 2: Table of hexadecimal digits and their decimal values (Byjus, n.d.).

3.4. XOR Logic Gate

XOR logic gate produces 1 only when the inputs are different, marking itself useful as a reversible mixing operation in LGCC (Geeksforgeeks, 2025).

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Table 3: XOR Gate truth table (Geeksforgeeks, 2025).

3.5. AND Logic Gate

AND logic gate only produces 1 as output when both the inputs are 1 as well (Geeksforgeeks, 2025).

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Table 4: AND gate truth table (Geeksforgeeks, 2025).

3.6. Encryption steps (per byte)

LGCC follows the following steps to encrypt.

1. Compute the shift value: $s_i = (K[i \bmod m] + i) \bmod 256$.
2. Select the gate byte: $g_i = K[(i + 1) \bmod m]$.
3. Derive the rotation amount using an AND mask: $r_i = K[(i + 2) \bmod m] \text{ AND } 7$.
The AND with 7 ensures r_i stays between 0 and 7 for an 8-bit rotation.
4. Apply Caesar-style shift: $t_i = (P_i + s_i) \bmod 256$
5. Apply XOR mixing: $u_i = t_i \text{ XOR } g_i$.
6. Apply chaining: $v_i = u_i \text{ XOR } \text{prev}$, where for $i = 0$, $\text{prev} = K[\text{byte}[0]]$ and for $i > 0$, $\text{prev} = C_{(i-1)}$.
7. Rotate left to produce ciphertext: $C_i = \text{ROTL8}(v_i, r_i)$.
8. Update prev.

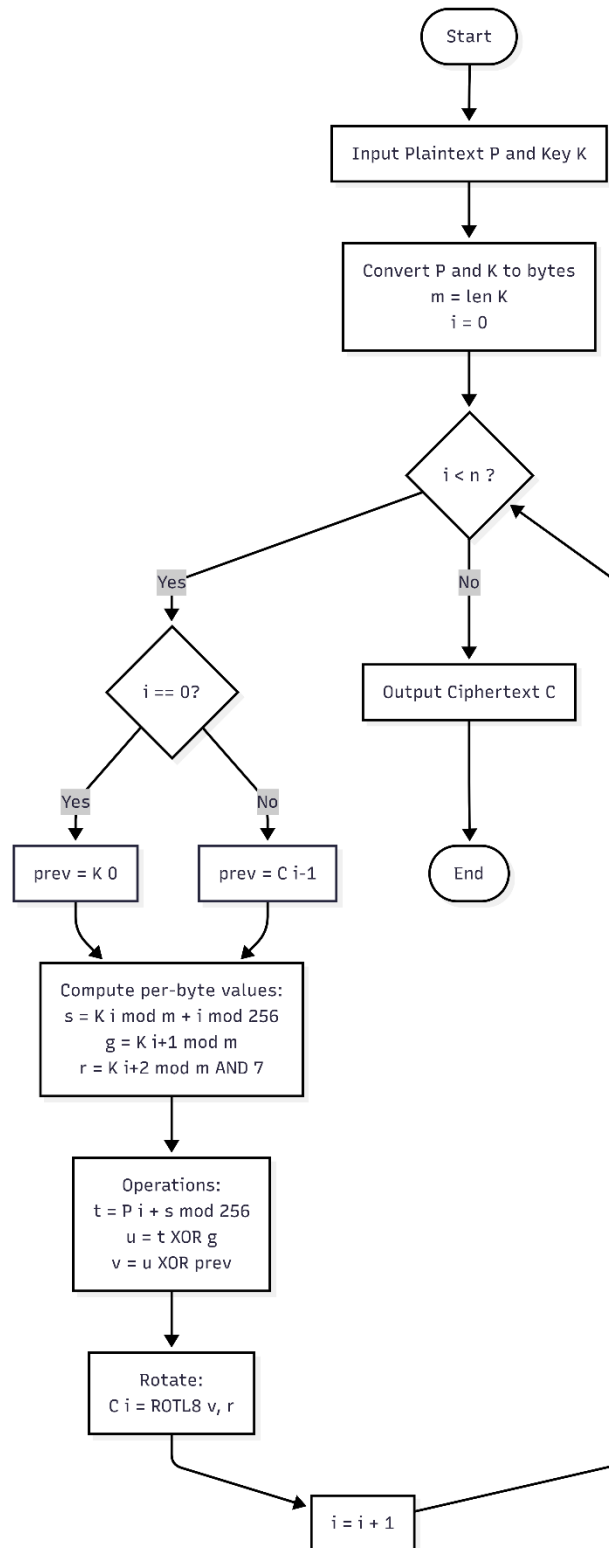


Figure 1: Flowchart of Encryption process.

3.7. Decryption steps (reverse process)

Decryption reverses encryption because each operation is invertible: a left rotation is reversed by a right rotation, XOR is its own inverse, and modular addition is reversed by modular subtraction.

1. Compute the same s_i , g_i , and r_i for index i .
2. Undo rotation: $v_i = \text{ROTR8}(C_i, r_i)$.
3. Undo chaining: $u_i = v_i \text{ XOR } \text{prev}$, where for $i = 0$, $\text{prev} = \text{Kbyte}[0]$ and for $i > 0$, $\text{prev} = C_{(i-1)}$.
4. Undo XOR mixing: $t_i = u_i \text{ XOR } g_i$.
5. Undo Caesar shift: $P_i = (t_i - s_i) \bmod 256$.
6. Update prev .

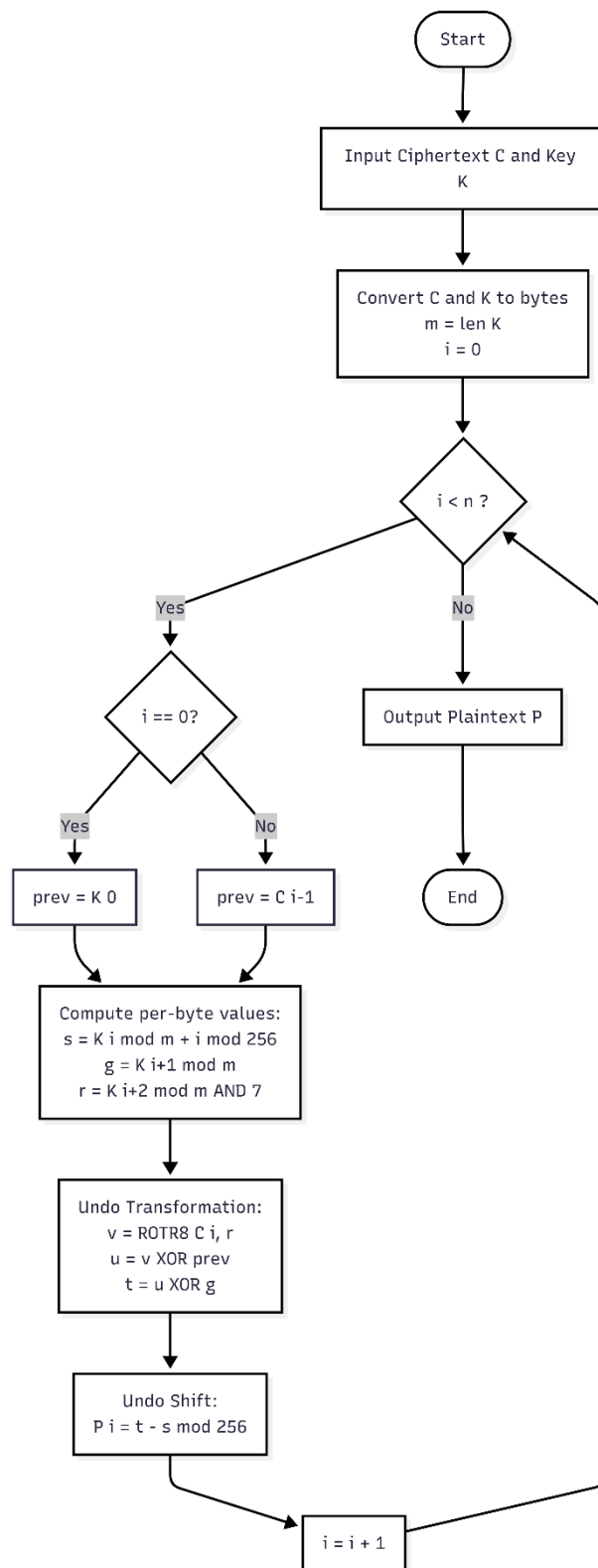


Figure 2: Flowchart of the Decryption process.

3.8. Algorithm explanation

This subsection provides a concise explanation of the Logic Gate Caesar Cipher (LGCC) and demonstrates the encryption process using a small worked example. LGCC operates on byte values (0–255). For each plaintext byte, a Caesar-style modular shift is applied, the result is mixed using XOR (a reversible logic-gate operation), ciphertext feedback is applied to reduce repeated patterns, and a small bit rotation is used to improve diffusion.

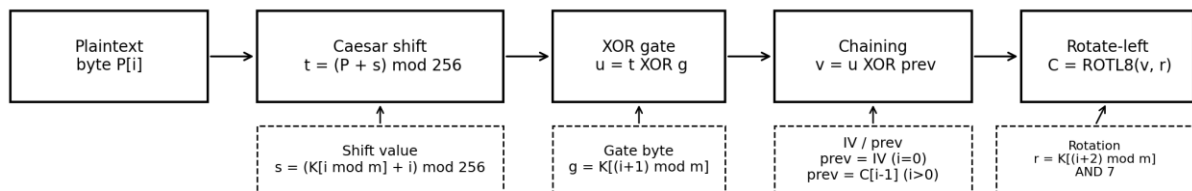


Figure 3: Per-byte processing pipeline used by LGCC, including the key-derived values that are recomputed for every index i .

Ciphertext feedback (also referred to as chaining in this design) uses the previous ciphertext byte as an input to the current encryption step. This makes repeated plaintext bytes less likely to produce repeated ciphertext bytes. Figure 2 illustrates how the prev value changes across successive bytes. LGCC uses XOR as a logic-gate mixing operation. LGCC applies an 8-bit left rotation as the final step of encryption for each byte.

Example of XOR mixing ($u = t \text{ XOR } g$)

$$\begin{array}{r}
 t = 0x93 = 10010011 \\
 g = 0x45 = 01000101 \\
 \text{XOR} \\
 \hline
 u = 0xD6 = 11010110
 \end{array}$$

Figure 4: XOR mixing.

3.9. Worked example for encryption

Example setup:

- Plaintext: "HELLO" → bytes: [48 45 4C 4C 4F] (hex)
- Key: "KEY" → bytes: [4B 45 59] (hex)
- $m = 3$ (length of key in bytes)
- for $i = 0$, $\text{prev} = \text{Kbyte}[0]$ and for $i > 0$, $\text{prev} = C_{(i-1)}$.

Now, from the formulae:

- $s_i = (K[i \bmod m] + i) \bmod 256$
- $g_i = K[(i + 1) \bmod m]$
- $r_i = K[(i + 2) \bmod m] \text{ AND } 7$
- $t_i = (P_i + s_i) \bmod 256$
- $u_i = t_i \text{ XOR } g_i$
- $v_i = u_i \text{ XOR } \text{prev}$
- $C_i = \text{ROTL8}(v_i, r_i)$

i	P	P (hex)	s (hex)	g (hex)	r	t (hex)	u (hex)	prev (hex)	v (hex)	C (hex)
0	H	0x48	0x4B	0x45	1	0x93	0xD6	0x4B	0x9D	0x3B
1	E	0x45	0x46	0x59	3	0x8B	0xD2	0x3B	0xE9	0x4F
2	L	0x4C	0x5B	0x4B	5	0xA7	0xEC	0x4F	0xA3	0x74
3	L	0x4C	0x4E	0x45	1	0x9A	0xDF	0x74	0xAB	0x57
4	O	0x4F	0x49	0x59	3	0x98	0xC1	0x57	0x96	0xB4

Table 5: Full encryption trace for HELLO using KEY.

After processing all five bytes, the ciphertext bytes are 3B 4F 74 57 B4 (hex). Therefore, the ciphertext (hex string) is 3b4f7457b4.

4. Task 04: Example test cases

This section validates the working steps of LGCC by showing 5+ example test cases, demonstrating that multiple PLAINTEXTS encrypt into different CIPHERTEXTS and decrypt back correctly using the reverse process (undo rotation → undo chaining → undo XOR → undo Caesar shift).

Common Parameters:

- Key: "KEY" (hex bytes: 4B 45 59)

4.1. Encryption Process

4.1.1. Test Case 1: "Samriddha"

Plaintext (UTF-8 bytes, hex): 53 61 6D 72 69 64 64 68 61

Encryption Table:

i	P	P (hex)	s (hex)	g (hex)	r	t (hex)	u (hex)	prev (hex)	v (hex)	C (hex)
0	S	0x53	0x4B	0x45	1	0x9E	0xDB	0x4B	0x90	0x21
1	a	0x61	0x46	0x59	3	0xA7	0xFE	0x21	0xDF	0xFE
2	m	0x6D	0x5B	0x4B	5	0xC8	0x83	0xFE	0x7D	0xAF
3	r	0x72	0x4E	0x45	1	0xC0	0x85	0xAF	0x2A	0x54
4	i	0x69	0x49	0x59	3	0xB2	0xEB	0x54	0xBF	0xFD
5	d	0x64	0x5E	0x4B	5	0xC2	0x89	0xFD	0x74	0x8E
6	d	0x64	0x51	0x45	1	0xB5	0xF0	0x8E	0x7E	0xFC
7	h	0x68	0x4C	0x59	3	0xB4	0xED	0xFC	0x11	0x88
8	a	0x61	0x61	0x4B	5	0xC2	0x89	0x88	0x01	0x20

Table 6: Encryption table of "Samriddha".

Ciphertext bytes (hex): 21 FE AF 54 FD 8E FC 88 20

Ciphertext (hex string): 21feaf54fd8efc8820

4.1.2. Test Case 02: “Security”

Plaintext (UTF-8 bytes, hex): 53 45 43 55 52 49 54 59

Encryption Table:

i	P	P (hex)	s (hex)	g (hex)	r	t (hex)	u (hex)	prev (hex)	v (hex)	C (hex)
0	S	0x53	0x4B	0x45	1	0x9E	0xDB	0x4B	0x90	0x21
1	E	0x45	0x46	0x59	3	0x8B	0xD2	0x21	0xF3	0x9F
2	C	0x43	0x5B	0x4B	5	0x9E	0xD5	0x9F	0x4A	0x49
3	U	0x55	0x4E	0x45	1	0xA3	0xE6	0x49	0xAF	0x5F
4	R	0x52	0x49	0x59	3	0x9B	0xC2	0x5F	0x9D	0xEC
5	I	0x49	0x5E	0x4B	5	0xA7	0xEC	0xEC	0x00	0x00
6	T	0x54	0x51	0x45	1	0xA5	0xE0	0x00	0xE0	0xC1
7	Y	0x59	0x4C	0x59	3	0xA5	0xFC	0xC1	0x3D	0xE9

Table 7: Encryption table of “Security”.

Ciphertext bytes (hex): 21 9F 49 5F EC 00 C1 E9

Ciphertext (hex string): 219f495fec00c1e9

4.1.3. Test Case 03: “CIA-TRIAD”

Plaintext (UTF-8 bytes, hex): 43 49 41 2D 54 52 49 41 44

Encryption Table:

i	P	P (hex)	s (hex)	g (hex)	r	t (hex)	u (hex)	prev (hex)	v (hex)	C (hex)
---	---	------------	------------	------------	---	------------	------------	---------------	------------	------------

0	C	0x43	0x4B	0x45	1	0x8E	0xCB	0x4B	0x80	0x01
1	I	0x49	0x46	0x59	3	0x8F	0xD6	0x01	0xD7	0xBE
2	A	0x41	0x5B	0x4B	5	0x9C	0xD7	0xBE	0x69	0x2D
3	-	0x2D	0x4E	0x45	1	0x7B	0x3E	0x2D	0x13	0x26
4	T	0x54	0x49	0x59	3	0x9D	0xC4	0x26	0xE2	0x17
5	R	0x52	0x5E	0x4B	5	0xB0	0xFB	0x17	0xEC	0x9D
6	I	0x49	0x51	0x45	1	0x9A	0xDF	0x9D	0x42	0x84
7	A	0x41	0x4C	0x59	3	0x8D	0xD4	0x84	0x50	0x82
8	D	0x44	0x61	0x4B	5	0xA5	0xEE	0x82	0x6C	0x8D

Table 8: Encryption table of "CIA-TRIAD".

Ciphertext bytes (hex): 01 BE 2D 26 17 9D 84 82 8D

Ciphertext (hex string): 01be2d26179d84828d

4.1.4. Test Case 04: "Attack at dawn!"

Plaintext (UTF-8 bytes, hex): 41 74 74 61 63 6B 20 61 74 20 64 61 77 6E 21

Encryption Table:

i	P	P (hex)	s (hex)	g (hex)	r	t (hex)	u (hex)	prev (hex)	v (hex)	C (hex)
0	A	0x41	0x4B	0x45	1	0x8C	0xC9	0x4B	0x82	0x05
1	t	0x74	0x46	0x59	3	0xBA	0xE3	0x05	0xE6	0x37
2	t	0x74	0x5B	0x4B	5	0xCF	0x84	0x37	0xB3	0x76
3	a	0x61	0x4E	0x45	1	0xAF	0xEA	0x76	0x9C	0x39
4	c	0x63	0x49	0x59	3	0xAC	0xF5	0x39	0xCC	0x66
5	k	0x6B	0x5E	0x4B	5	0xC9	0x82	0x66	0xE4	0x9C
6		0x20	0x51	0x45	1	0x71	0x34	0x9C	0xA8	0x51
7	a	0x61	0x4C	0x59	3	0xAD	0xF4	0x51	0xA5	0x2D

8	t	0x74	0x61	0x4B	5	0xD5	0x9E	0x2D	0xB3	0x76
9		0x20	0x54	0x45	1	0x74	0x31	0x76	0x47	0x8E
10	d	0x64	0x4F	0x59	3	0xB3	0xEA	0x8E	0x64	0x23
11	a	0x61	0x64	0x4B	5	0xC5	0x8E	0x23	0xAD	0xB5
12	w	0x77	0x57	0x45	1	0xCE	0x8B	0xB5	0x3E	0x7C
13	n	0x6E	0x52	0x59	3	0xC0	0x99	0x7C	0xE5	0x2F
14	!	0x21	0x67	0x4B	5	0x88	0xC3	0x2F	0xEC	0x9D

Table 9: Encryption table of "Attack at dawn!".

Ciphertext bytes (hex): 05 37 76 39 66 9C 51 2D 76 8E 23 B5 7C 2F 9D

Ciphertext (hex string): 05377639669c512d768e23b57c2f9d

4.1.5. Test Case 05: "12345"

Plaintext (UTF-8 bytes, hex): 31 32 33 34 35

Encryption Table:

i	P	P (hex)	s (hex)	g (hex)	r	t (hex)	u (hex)	prev (hex)	v (hex)	C (hex)
0	1	0x31	0x4B	0x45	1	0x7C	0x39	0x4B	0x72	0xE4
1	2	0x32	0x46	0x59	3	0x78	0x21	0xE4	0xC5	0x2E
2	3	0x33	0x5B	0x4B	5	0x8E	0xC5	0x2E	0xEB	0x7D
3	4	0x34	0x4E	0x45	1	0x82	0xC7	0x7D	0xBA	0x75
4	5	0x35	0x49	0x59	3	0x7E	0x27	0x75	0x52	0x92

Table 10: Encryption table of "12345".

Ciphertext bytes (hex): E4 2E 7D 75 92

Ciphertext (hex string): e42e7d7592

4.2. Decryption Process

4.2.1. Test Case 1: “Samriddha”

Ciphertext bytes (hex): 21 FE AF 54 FD 8E FC 88 20

Decryption Table:

i	C (hex)	r	v=ROTR8(C,r) (hex)	prev (hex)	u=v XOR prev (hex)	g (hex)	t=u XOR g (hex)	s (hex)	P (hex)	P
0	0x21	1	0x90	0x4B	0xDB	0x45	0x9E	0x4B	0x53	S
1	0xFE	3	0xDF	0x21	0xFE	0x59	0xA7	0x46	0x61	a
2	0xAF	5	0x7D	0xFE	0x83	0x4B	0xC8	0x5B	0x6D	m
3	0x54	1	0x2A	0xAF	0x85	0x45	0xC0	0x4E	0x72	r
4	0xFD	3	0xBF	0x54	0xEB	0x59	0xB2	0x49	0x69	i
5	0x8E	5	0x74	0xFD	0x89	0x4B	0xC2	0x5E	0x64	d
6	0xFC	1	0x7E	0x8E	0xF0	0x45	0xB5	0x51	0x64	d
7	0x88	3	0x11	0xFC	0xED	0x59	0xB4	0x4C	0x68	h
8	0x20	5	0x01	0x88	0x89	0x4B	0xC2	0x61	0x61	a

Table 11: Decryption Table of "Samriddha".

Decryption output: Samriddha

4.2.2. Test Case 02: “Security”

Ciphertext bytes (hex): 21 9F 49 5F EC 00 C1 E9

Decryption Table:

i	C (hex)	r	v=ROTR8(C,r) (hex)	prev (hex)	u=v XOR prev (hex)	g (hex)	t=u XOR g (hex)	s (hex)	P (hex)	P

0	0x21	1	0x90	0x4B	0xDB	0x45	0x9E	0x4B	0x53	S
1	0x9F	3	0xF3	0x21	0xD2	0x59	0x8B	0x46	0x45	E
2	0x49	5	0x4A	0x9F	0xD5	0x4B	0x9E	0x5B	0x43	C
3	0x5F	1	0xAF	0x49	0xE6	0x45	0xA3	0x4E	0x55	U
4	0xEC	3	0x9D	0x5F	0xC2	0x59	0x9B	0x49	0x52	R
5	0x00	5	0x00	0xEC	0xEC	0x4B	0xA7	0x5E	0x49	I
6	0xC1	1	0xE0	0x00	0xE0	0x45	0xA5	0x51	0x54	T
7	0xE9	3	0x3D	0xC1	0xFC	0x59	0xA5	0x4C	0x59	Y

Table 12: Decryption Table of "Security".

Decryption output: SECURITY

4.2.3. Test Case 03: "CIA-TRIAD"

Ciphertext bytes (hex): 01 BE 2D 26 17 9D 84 82 8D

Decryption Table:

i	C (hex)	r	v=ROTR8(C,r) (hex)	prev (hex)	u=v XOR prev (hex)	g (hex)	t=u XOR g (hex)	s (hex)	P (hex)	P
0	0x01	1	0x80	0x4B	0xCB	0x45	0x8E	0x4B	0x43	C
1	0xBE	3	0xD7	0x01	0xD6	0x59	0x8F	0x46	0x49	I
2	0x2D	5	0x69	0xBE	0xD7	0x4B	0x9C	0x5B	0x41	A
3	0x26	1	0x13	0x2D	0x3E	0x45	0x7B	0x4E	0x2D	-
4	0x17	3	0xE2	0x26	0xC4	0x59	0x9D	0x49	0x54	T
5	0x9D	5	0xEC	0x17	0xFB	0x4B	0xB0	0x5E	0x52	R
6	0x84	1	0x42	0x9D	0xDF	0x45	0x9A	0x51	0x49	I
7	0x82	3	0x50	0x84	0xD4	0x59	0x8D	0x4C	0x41	A

8	0x8D	5	0x6C	0x82	0xEE	0x4B	0xA5	0x61	0x44	D
---	------	---	------	------	------	------	------	------	------	---

Table 13: Decryption Table of "CIA-TRIAD".

Decryption output: CIA-TRIAD

4.2.4. Test Case 04: "Attack at dawn!"

Ciphertext bytes (hex): 05 37 76 39 66 9C 51 2D 76 8E 23 B5 7C 2F 9D

Decryption Table:

i	C (hex)	r	v=ROTR8(C,r) (hex)	prev (hex)	u=v XOR prev (hex)	g (hex)	t=u XOR g (hex)	s (hex)	P (hex)	P
0	0x05	1	0x82	0x4B	0xC9	0x45	0x8C	0x4B	0x41	A
1	0x37	3	0xE6	0x05	0xE3	0x59	0xBA	0x46	0x74	t
2	0x76	5	0xB3	0x37	0x84	0x4B	0xCF	0x5B	0x74	t
3	0x39	1	0x9C	0x76	0xEA	0x45	0xAF	0x4E	0x61	a
4	0x66	3	0xCC	0x39	0xF5	0x59	0xAC	0x49	0x63	c
5	0x9C	5	0xE4	0x66	0x82	0x4B	0xC9	0x5E	0x6B	k
6	0x51	1	0xA8	0x9C	0x34	0x45	0x71	0x51	0x20	
7	0x2D	3	0xA5	0x51	0xF4	0x59	0xAD	0x4C	0x61	a
8	0x76	5	0xB3	0x2D	0x9E	0x4B	0xD5	0x61	0x74	t
9	0x8E	1	0x47	0x76	0x31	0x45	0x74	0x54	0x20	
10	0x23	3	0x64	0x8E	0xEA	0x59	0xB3	0x4F	0x64	d
11	0xB5	5	0xAD	0x23	0x8E	0x4B	0xC5	0x64	0x61	a
12	0x7C	1	0x3E	0xB5	0x8B	0x45	0xCE	0x57	0x77	w
13	0x2F	3	0xE5	0x7C	0x99	0x59	0xC0	0x52	0x6E	n
14	0x9D	5	0xEC	0x2F	0xC3	0x4B	0x88	0x67	0x21	!

Table 14: Decryption Table of "Attack at dawn!".

Decryption output: Attack at dawn!

4.2.5. Test Case 05: “12345”

Ciphertext bytes (hex): E4 2E 7D 75 92

Decryption Table:

i	C (hex)	r	v=ROTR8(C,r) (hex)	prev (hex)	u=v XOR prev (hex)	g (hex)	t=u XOR g (hex)	s (hex)	P (hex)	P
0	0xE4	1	0x72	0x4B	0x39	0x45	0x7C	0x4B	0x31	1
1	0x2E	3	0xC5	0xE4	0x21	0x59	0x78	0x46	0x32	2
2	0x7D	5	0xEB	0x2E	0xC5	0x4B	0x8E	0x5B	0x33	3
3	0x75	1	0xBA	0x7D	0xC7	0x45	0x82	0x4E	0x34	4
4	0x92	3	0x52	0x75	0x27	0x59	0x7E	0x49	0x35	5

Table 15: Decryption Table of "12345".

Decryption output: 12345

5. Task 05: Critical Evaluation of the LGCC Algorithm

This section critically evaluates the strengths and weaknesses of the newly designed Logic Gate Caesar Cipher (LGCC) created in Task 03. LGCC is a symmetric byte-based cipher that combines:

1. Caesar-style modular addition,
2. XOR “logic-gate” mixing,
3. Ciphertext feedback/chaining,
4. An 8-bit rotation to improve diffusion,

while remaining reversible through the defined decryption steps.

5.1. Strengths of LGCC

1. Reversibility is formally guaranteed

LGCC is built from invertible operations: ROTL/ROTR, XOR (self-inverse), and modular addition/subtraction, so the cipher can reliably decrypt to the original plaintext using the reverse pipeline.

2. Works on full byte range (0–255), supporting real text encodings

Because LGCC operates on bytes rather than only A–Z letters, it can process general UTF-8/ASCII byte data instead of being limited to alphabetic Caesar shifts.

3. Improved diffusion through 8-bit rotation

The final per-byte rotation spreads bit changes across positions, so a one-bit change in intermediate values can affect different output bits, improving diffusion compared to plain substitution-only Caesar.

4. Reduces repeated-pattern leakage via ciphertext feedback (chaining)

By mixing the previous ciphertext byte into the current encryption step (feedback), repeated plaintext bytes become less likely to produce repeated ciphertext bytes, addressing a major weakness of simple Caesar substitution.

5. Efficient, lightweight operations (good for learning/implementation clarity)

LGCC uses basic integer addition/subtraction, XOR, AND masking, and bit rotation—operations that are fast and simple to implement and explain, aligning with the stated goal of keeping a Caesar-based design understandable while adding extra hardening steps.

6. Key-driven variation per index i

The shift value s_i , gate byte g_i , and rotation amount r_i are recomputed for each byte index i , so processing is not a single fixed substitution across the whole message.

5.2. Weaknesses of LGCC

1. Not a standardized or peer-reviewed cipher

LGCC is a newly created design; without formal cryptanalysis and public review, its real security level against modern attacks cannot be assumed (even if it “looks complex”). This is an inherent risk for any custom cipher.

2. Key schedule is simple and repeats every key length m

Because s_i , g_i , and r_i depend on i modulus m , the key material is reused in a repeating pattern across the message. This repetition can leak structure over long messages and can make analysis easier than ciphers with stronger key expansion.

3. Operations are mostly linear/structured (XOR, rotation, modular add)

XOR and bit-rotation are structured operations, and Caesar-style shifting is also highly regular. Your pipeline is reversible (good) but those same properties can be exploited in known-plaintext or differential-style reasoning if the design lacks stronger nonlinearity.

4. Chaining is only 1-byte feedback (limited avalanche across the whole message)

LGCC uses the previous ciphertext byte as feedback. This helps with repeated symbols, but the dependency is local (only on the previous byte), so a change may not strongly avalanche through the entire ciphertext compared to multi-round designs.

5. Rotation amount has limited range (0–7)

The AND mask with 7 forces r_i into 0–7. This restriction reduces variability: the diffusion step is present, but the rotation control space is small and derived directly from repeating key bytes.

6. No built-in integrity/authentication

LGCC emphasizes confidentiality (encryption/decryption). It does not implement a means to recognize alterations to the ciphertext; as a result, modified ciphertext could unencrypt to bad plaintext, and there would be no way to identify this (a frequent shortcoming of encryption-only plans). This is relevant in the context of secure messaging in the real world.

From a modern cryptographic perspective, LGCC is a custom design and is not backed by formal security proofs (e.g., indistinguishability under chosen-plaintext attack). In addition, LGCC provides confidentiality only and does not include authentication, meaning ciphertext tampering may not be detectable. Therefore, LGCC is best treated as an educational cipher demonstrating diffusion and feedback rather than a replacement for standardized algorithms.

5.3. Application Area of LGCC

LGCC is best positioned as an educational / demonstrative cipher because it is a Caesar-based design extended with logic-gate mixing, rotation, and chaining to reduce patterns while remaining understandable and reversible.

Practical application areas include:

- Teaching cryptography fundamentals (showing how Caesar weakness can be reduced using XOR, feedback, and diffusion).
- Simple data obfuscation in low-stakes contexts where clarity and reversibility matter more than high-assurance security.
- Learning implementations (students practicing byte operations, hex representations, and invertible transforms).

6. Conclusion

This report introduced core security concepts and the CIA triad, then explained cryptography fundamentals and contrasted symmetric and asymmetric encryption. It highlighted the Caesar cipher's main weaknesses like small key space and visible frequency/pattern leakage and justified the need for modification using key-based variation, mixing, diffusion, and feedback. Based on this rationale, the Logic Gate Caesar Cipher (LGCC) was developed as a symmetric byte-oriented algorithm using modular shifting, XOR logic-gate mixing, ciphertext feedback, and 8-bit rotation, alongside a defined reverse decryption process to recover plaintext. The algorithm's correctness was demonstrated through worked examples and multiple test cases where different plaintexts produced different ciphertexts and decrypted back successfully. Finally, a critical evaluation showed that while LGCC improves diffusion and reduces simple repetition patterns compared to a basic Caesar cipher, it remains a custom design with structural limitations and is most appropriate for educational and learning-oriented use.

7. References

- Britannica. (2025, December 19). *Caesar Cipher*. Retrieved from Britannica : <https://www.britannica.com/topic/Caesar-cipher>
- Britannica. (2025). *data encryption* . Retrieved from Britannica : <https://www.britannica.com/technology/data-encryption>
- Britannica. (2025, December 5). *History of Cryptography* . Retrieved from Britannica : <https://www.britannica.com/topic/cryptology/History-of-cryptology>
- Byjus. (n.d.). *Hexadecimal Number System*. Retrieved from Byjus: <https://byjus.com/maths/hexadecimal-number-system/>
- Geeksforgeeks. (2025, November 22). *AND Gate*. Retrieved from Geeksforgeeks: <https://www.geeksforgeeks.org/digital-logic/and-gate/>
- Geeksforgeeks. (2025, July 23). *XOR Gate*. Retrieved from Geeksforgeeks: <https://www.geeksforgeeks.org/digital-logic/xor-gate/>
- National Institute of Standards and Technology (NIST). (n.d.). *Public key cryptography (PKC)*. Retrieved from NIST: https://csrc.nist.gov/glossary/term/public_key_cryptography
- National Institute of Standards and Technology (NIST). (n.d.). *Symmetric Cryptography*. Retrieved from NIST: https://csrc.nist.gov/glossary/term/Symmetric_Cryptography
- National Institute of Standards and Technology (NIST). (n.d.). *Encryption Glossary* . Retrieved from NIST: <https://csrc.nist.gov/glossary/term/encryption>
- National Institute of Standards and Technology (NIST). (2017). *NIST Special Publication 800-12 Revision 1: An Introduction to Information Security*. NIST.
- National Institute of Standards and Technology (NIST). (2004, July 5). *special Publication 800-12: An Introduction to Computer Security: The NIST Handbook*. Retrieved from NIST: <https://csrc.nist.gov/publications/nistpubs/800-12/800-12-html/chapter1.html>
- National Institute of Standards and Technology (NIST). (n.d.). *Computer Security Resource Center*. Retrieved from NIST: <https://csrc.nist.gov/glossary/term/cryptography>
- Rapidtables. (n.d.). *ASCII Table*. Retrieved from Rapidtables: <https://www.rapidtables.com/code/text/ascii-table.html>

8. Appendices

8.1. Appendix A: LGCC Supporting Artefacts

8.1.1. Variable definition

The LGCC algorithm processes plaintext as bytes and generates ciphertext as bytes. For each byte index i :

- K is the key as UTF-8 bytes, length m (minimum 3 bytes; if shorter, pad with 00 until $m=3$).
- P_i is the plaintext byte at index i .
- C_i is the ciphertext byte at index i .
- $s_i = (K[i \bmod m] + i) \bmod 256$
- $g_i = K[(i + 1) \bmod m]$
- $r_i = K[(i + 2) \bmod m] \text{ AND } 7$ (rotation count in range 0–7)
- prev is the chaining value: where for $i = 0$, $\text{prev} = K\text{byte}[0]$ and for $i > 0$, $\text{prev} = C_{(i-1)}$.

8.1.2. Bit rotation functions

- $\text{ROTL8}(x, r)$ rotates 8-bit value x left by r bits.
- $\text{ROTR8}(x, r)$ rotates 8-bit value x right by r bits.

All intermediate results are constrained to 0–255.

8.2. Appendix B: LGCC Pseudocode

8.2.1. Encryption and Decryption Pseudocode

$\text{LGCC_Encrypt}(P, K)$:

$K\text{bytes} = \text{UTF8_bytes}(K)$

$m = \text{len}(K\text{bytes})$

if $m < 3$:

 pad $K\text{bytes}$ with 00 until length 3

m = 3

n = len(P)

prev = Kbytes[0] // initial chaining value for i = 0

for i from 0 to n-1:

 s = (Kbytes[i mod m] + i) mod 256

 g = Kbytes[(i+1) mod m]

 r = Kbytes[(i+2) mod m] AND 7

 t = (P[i] + s) mod 256 // Caesar base (byte shift)

 u = t XOR g // logic gate mixing

 v = u XOR prev // chaining (feedback)

 C[i] = ROTL8(v, r) // diffusion

 prev = C[i] // feedback update for next round

return C

LGCC_Decrypt(C, K):

 Kbytes = UTF8_bytes(K)

 m = len(Kbytes)

 if m < 3:

 pad Kbytes with 00 until length 3

 m = 3

```
n = len(C)
prev = Kbytes[0]          // same initial chaining value for i = 0

for i from 0 to n-1:
    s = (Kbytes[i mod m] + i) mod 256
    g = Kbytes[(i+1) mod m]
    r = Kbytes[(i+2) mod m] AND 7

    v = ROTR8(C[i], r)     // undo rotation
    u = v XOR prev        // undo chaining
    t = u XOR g           // undo mixing
    P[i] = (t - s) mod 256 // undo Caesar shift

    prev = C[i]           // feedback update for next round

return P
```